Document Loading & Text Splitting

LangChain v1.2.x — Section 07

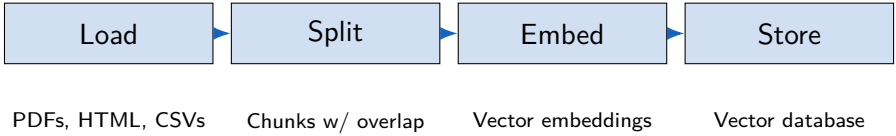
AI Engineering

March 2026

Section Overview

- **Why it matters:** Data ingestion is the foundation of RAG pipelines
- **Learning objectives:**
  - Understand the Document ingestion pipeline
  - Load documents from various sources
  - Split documents intelligently for retrieval
  - Manage metadata through the pipeline
- **Scope:** LangChain v1.2.x (March 2026)
- **Slides:** 40 slides covering theory, code, and best practices

The Data Ingestion Pipeline



- **Load:** Retrieve raw documents from various sources
- **Split:** Break documents into manageable chunks
- **Embed:** Convert text to vector representations
- **Store:** Index vectors for efficient retrieval

The Document Object

```python
from langchain_core.documents import Document
doc = Document(
  page_content="Some text",
  metadata={"source": "example.pdf", "page": 1}
)
print(doc.page_content)
```

Key Aspects

- `page_content`: Actual text
- `metadata`: Arbitrary key-value pairs
- Metadata preserved through pipeline

Document Loaders — Common Interface

All loaders inherit from BaseLoader:

```python
from langchain_core.document_loaders import BaseLoader
from langchain_core.documents import Document
from typing import List

class BaseLoader(BaseModel):
    def load(self) -> List[Document]:
        """Load documents and return as list"""
        raise NotImplementedError

    def lazy_load(self) -> Iterator[Document]:
        """Stream documents one at a time"""
        raise NotImplementedError
```

Two Load Modes

- `load()`: Return all documents at once (for small datasets)
- `lazy_load()`: Stream documents (for large datasets, memory efficient)

TextLoader — Plain Text Files

Load simple text files:

```python
from langchain_community.document_loaders import TextLoader

loader = TextLoader("document.txt")
documents = loader.load()

# For large files, stream instead:
for doc in loader.lazy_load():
    print(doc.page_content[:100])
```

Use Cases

- .txt, .log, configuration files
- Source code files
- Simple documents without formatting

[Docs: TextLoader: langchain/document_loaders/text]

PyPDFLoader — PDF Documents

Extract text from PDF files with page tracking:

```python
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("document.pdf")
documents = loader.load()

# Each page becomes a Document
for doc in documents:
    print(f"Page {doc.metadata['page']}: {doc.page_content[:100]}")
```

- Automatically extracts metadata (page number, file path)
- One `Document` per PDF page
- Preserves text structure from PDF

[Docs: PyPDFLoader: langchain_community/document_loaders/pdf]

CSVLoader — Tabular Data

Load CSV files as documents:

```
from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader(
    file_path="data.csv",
    source_column="name"  # Which column is the source
)
documents = loader.load()

# Each row becomes a Document
for doc in documents:
    print(doc.page_content)
```

- Converts each CSV row to structured text
- Can specify which column contains the "source"
- Useful for structured data like product catalogs

[Docs: CSVLoader: langchain_community/document_loaders/csv]

WebBaseLoader — Web Pages

Scrape and load web pages:

```
from langchain_community.document_loaders import WebBaseLoader

urls = [
    "https://example.com/page1",
    "https://example.com/page2"
]
loader = WebBaseLoader(urls)
documents = loader.load()

for doc in documents:
    print(f"Source: {doc.metadata['source']}")
    print(f"Content: {doc.page_content[:100]}")
```

- Strips HTML, keeps text content
- Preserves source URL in metadata
- Great for knowledge base ingestion

[Docs: WebBaseLoader: langchain_community/document_loaders/web]

DirectoryLoader — Batch File Loading

Load multiple files from a directory:

```
from langchain_community.document_loaders import DirectoryLoader

loader = DirectoryLoader(
    path="./documents/",
    glob="**/*.txt",  # Glob pattern for filtering
    loader_cls=TextLoader
)
documents = loader.load()
print(f"Loaded {len(documents)} documents")
```

Features

- Use glob patterns to filter files
- Specify which loader class to use
- Supports recursive directory traversal (**)

[Docs: DirectoryLoader: langchain_community/document_loaders/directory]

Document Loaders — Comparison

Loader	Source	Metadata	Streaming	Notes
TextLoader	.txt files	filename	Yes	Simple, fast
PyPDFLoader	PDFs	page #, path	Yes	Per-page split
CSVLoader	CSV	row-based	Yes	Structured data
WebBaseLoader	URLs	source URL	Yes	HTML stripping
DirectoryLoader	Directory	per-file	Yes	Batch processing
JSONLoader	JSON	custom	Yes	Structured
MarkdownLoader	.md	path, heading	Yes	Code-friendly

All loaders support both `load()` and `lazy_load()` modes

Custom Loaders — Implementing BaseLoader

Create a custom loader for proprietary formats:

```
from langchain_core.document_loaders import BaseLoader
from langchain_core.documents import Document
from typing import List

class CustomLoader(BaseLoader):
    def __init__(self, file_path: str):
        self.file_path = file_path

    def load(self) -> List[Document]:
        with open(self.file_path, 'r') as f:
            content = f.read()
        return [Document(
            page_content=content,
            metadata={"source": self.file_path}
        )]
```

Use: `docs = CustomLoader("data.custom").load()`

Why Text Splitting Matters

- **Context windows are limited**: LLMs have finite token limits
  - GPT-4: 8K, 32K, or 128K tokens
  - Claude: 100K+ tokens
  - Splitting ensures chunks fit in context
- **Retrieval quality**: Small, focused chunks improve semantic search
- **Cost efficiency**: Pay for what you retrieve, not entire documents
- **Relevance**: Optimal chunk size balances specificity and context

The Splitting Dilemma Too small: Lose context. Too large: Exceed token limits or retrieve irrelevant content.

RecursiveCharacterTextSplitter — The Default

Most commonly used splitter in production:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # Max characters per chunk
    chunk_overlap=200,        # Characters to overlap
    separators=[
        "\n\n",   # Paragraph
        "\n",     # Newline
        " ",      # Space
        ""        # Character fallback
    ]
)
chunks = splitter.split_documents(documents)
```

[Docs: RecursiveCharacterTextSplitter: langchain_text_splitters]

RecursiveCharacterTextSplitter — Separator Hierarchy

Splits recursively using a hierarchy of separators:

1. Try splitting on `"\n\n"` (paragraphs)
2. If chunks still too large, split on `"\n"` (lines)
3. If still too large, split on `" "` (words)
4. If still too large, split on `""` (characters)

Why Recursive?

- Preserves semantic structure (paragraphs → lines → words)
- Better context preservation than naive splitting
- Handles documents with mixed formatting

Chunk Size and Overlap — How to Choose

- **chunk_size** (characters or tokens):
  - Typical range: 512 to 2048 characters
  - For dense docs: 500-800 chars (3-5 sentences)
  - For sparse docs: 1000-2000 chars (longer context)
  - Rule of thumb: about 1/4 of model's context window
- **chunk_overlap** (characters or tokens):
  - Typical: 10-25 percent of chunk_size
  - Preserves context across chunk boundaries
  - 200 overlap on 1000 chunk = 20 percent
  - Too much overlap: redundant storage; too little: lost context

Tip Start with chunk_size=1000, overlap=200, then tune based on retrieval quality metrics.

Other Text Splitters

```python
# CharacterTextSplitter - simple, non-recursive
from langchain_text_splitters import CharacterTextSplitter
splitter = CharacterTextSplitter(chunk_size=1000, separator=" ")

# TokenTextSplitter - split by token count
from langchain_text_splitters import TokenTextSplitter
splitter = TokenTextSplitter(encoding_name="cl100k_base", chunk_size=100)

# MarkdownHeaderTextSplitter - preserve markdown structure
from langchain_text_splitters import MarkdownHeaderTextSplitter
splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "Header 1"), ("##", "Header 2")]
)
chunks = splitter.split_text(markdown_string)
```

Language-Aware Code Splitting

Split source code while preserving structure:

```
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=1000,
    chunk_overlap=200
)

with open("script.py", "r") as f:
    code = f.read()

chunks = splitter.split_text(code)
```

- Supports: Python, JavaScript, TypeScript, Java, C++, Go, Rust, and more
- Uses language-specific separators (imports, class definitions, functions)
- Critical for code QA and documentation systems

Preserving and Enriching Metadata

Metadata flows through the pipeline:

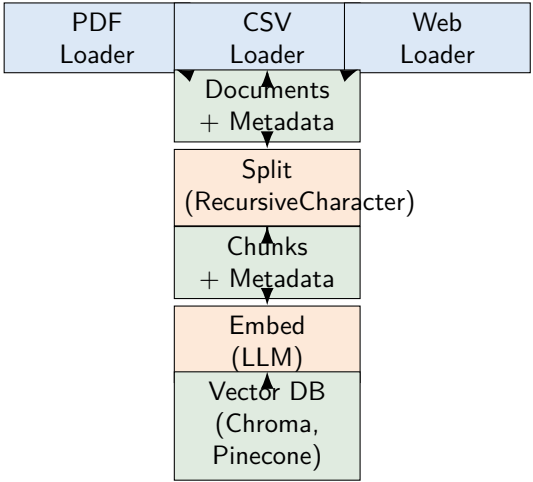
```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("doc.pdf")
docs = loader.load()  # Includes page metadata

splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
chunks = splitter.split_documents(docs)  # Preserves metadata!

# Enrich metadata
for i, chunk in enumerate(chunks):
    chunk.metadata["chunk_id"] = i
    chunk.metadata["chunk_order"] = i

# Use in retrieval
for chunk in chunks:
    print(f"Source: {chunk.metadata['source']}, "
          f"Page: {chunk.metadata['page']}")
```

Metadata Best Practices

- **Always preserve original metadata:**
  - Source file/URL
  - Page/section number
  - Author, timestamp
- **Add processing metadata:**
  - Chunk ID, chunk order
  - Loader class name
  - Processing timestamp
- **Use for traceability:**
  - Link retrieved chunks back to source
  - Support citations in LLM responses
  - Debug retrieval quality issues

Rule: Metadata must survive the entire pipeline: Load $\rightarrow$ Split $\rightarrow$ Embed $\rightarrow$ Store.

Document Processing Pipeline — Detailed View



Common Pitfall 1: Incorrect Chunk Size

Chunk Size	Problem	Impact
Too small (¡ 200)	Lose context	Poor LLM responses
Too large (¿ 4000)	Exceed token limits	API errors, cost
Not tuned	Varies by document	Inconsistent quality

Solution

- Estimate model's usable context: $0.25 \times$ context_window
- Start with default 1000, measure retrieval quality
- Use metrics: relevance score, response coherence, latency

Common Pitfall 2: Losing Metadata

Danger If metadata is lost during splitting, you cannot trace retrieved chunks back to source!

```
# WRONG - metadata lost
chunks = splitter.split_text(text)  # str input, no metadata

# CORRECT - metadata preserved
docs = [Document(page_content=text, metadata={...})]
chunks = splitter.split_documents(docs)  # Document input
```

Always use: `split_documents()` with Document objects, not `split_text()`.

Common Pitfall 3: Text Encoding Issues

Danger PDFs and web pages may contain special characters, encodings that cause corruption.

```
# Specify encoding when loading text files
from langchain_community.document_loaders import TextLoader

loader = TextLoader("document.txt", encoding="utf-8")
docs = loader.load()

# For PDFs, PyPDFLoader handles encoding automatically
# But test output for corrupted text
for doc in docs:
    if "\ufffd" in doc.page_content:  # Replacement char
        print("Warning: encoding error detected")
```

Tip: Always inspect a sample of loaded documents for corruption.

Full Example: Load → Split → Store

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# Load
loader = PyPDFLoader("document.pdf")
docs = loader.load()

# Split
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_documents(docs)

# Embed & Store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)

# Retrieve
results = vectorstore.similarity_search("query text", k=3)
```

Handling Large Documents Efficiently

Use streaming (lazy_load) for large files:

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("huge_document.pdf")
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)

# Process one document at a time
for doc in loader.lazy_load():
    chunks = splitter.split_documents([doc])
    # Process/embed/store chunks immediately
    vectorstore.add_documents(chunks)
    # Don't accumulate in memory!
```

- Avoids loading entire document into memory
- Process and store in batches
- Critical for PDFs with hundreds of pages

Batch Processing Multiple Files

```python
from langchain_community.document_loaders import DirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Load all PDFs in directory
loader = DirectoryLoader(
    "documents/",
    glob="**/*.pdf",
    loader_cls=PyPDFLoader
)

# Split and embed
splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
embeddings = OpenAIEmbeddings()
vectorstore = Chroma()

for doc in loader.lazy_load():
    chunks = splitter.split_documents([doc])
    vectorstore.add_documents(chunks)
```

Token-Based Splitting (More Accurate)

For precise control, split by token count instead of characters:

```
from langchain_text_splitters import TokenTextSplitter
from langchain.callbacks import get_openai_callback

# Split by tokens (more accurate than chars)
splitter = TokenTextSplitter(
    encoding_name="cl100k_base",  # GPT-4/3.5 encoding
    chunk_size=100,               # tokens
    chunk_overlap=20              # tokens
)

with get_openai_callback() as cb:
    chunks = splitter.split_documents(docs)
    print(f"Total tokens: {cb.total_tokens}")
```

Encoding names:

- `cl100k_base`: GPT-4, GPT-3.5-turbo
- `p50k_base`: GPT-3
- `r50k_base`: Text-davinci-003

Advanced: Custom Metadata Extraction

Add smart metadata during processing:

```
from datetime import datetime
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("document.pdf")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
chunks = splitter.split_documents(docs)

# Enrich with processing metadata
for i, chunk in enumerate(chunks):
    chunk.metadata.update({
        "chunk_sequence": i,
        "processed_at": datetime.now().isoformat(),
        "chunk_hash": hash(chunk.page_content) % (10**8),
        "character_count": len(chunk.page_content)
    })
```

Debugging Document Processing

```python
# Inspect loaded documents
docs = loader.load()
print(f"Loaded {len(docs)} documents")
for i, doc in enumerate(docs[:2]):
    print(f"Doc {i}: {len(doc.page_content)} chars")
    print(f"  Metadata: {doc.metadata}")

# Inspect chunks after splitting
chunks = splitter.split_documents(docs)
print(f"Created {len(chunks)} chunks")
for i, chunk in enumerate(chunks[:3]):
    print(f"Chunk {i}: {len(chunk.page_content)} chars, "
          f"tokens ~= {len(chunk.page_content) // 4}")
    print(f"  Preview: {chunk.page_content[:50]}...")

# Verify metadata preservation
if not all(chunks[0].metadata for chunk in chunks):
    print("WARNING: Some chunks missing metadata!")
```

Performance Tuning

- **Loading:**
  - Use `lazy_load()` for large files
  - Batch processing with `DirectoryLoader`
- **Splitting:**
  - `RecursiveCharacterTextSplitter` is fast (CPU only)
  - `TokenTextSplitter` slower (requires tokenizer)
  - Larger overlap = more chunks (exponential storage cost)
- **Embedding:**
  - Batch API calls (embed multiple chunks at once)
  - Cache embeddings to avoid recomputing
- **Storage:**
  - Use vector DB pagination for large collections
  - Monitor memory usage with `lazy_load()`

Special Case: Markdown with Headers

Preserve document structure with markdown-aware splitting:

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

markdown_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[
        ("#", "Header 1"),
        ("##", "Header 2"),
        ("###", "Header 3")
    ]
)

# Split markdown, preserving structure
chunks = markdown_splitter.split_text(markdown_content)

# Each chunk has header context in metadata
for chunk in chunks:
    print(f"Headers: {chunk.metadata}")
    print(f"Content: {chunk.page_content[:100]}")
```

Special Case: JSON Documents

Load and split structured JSON:

```
from langchain_community.document_loaders import JSONLoader
import json

# Load JSON file
loader = JSONLoader(
    file_path="data.json",
    jq_schema=".[]"  # Extract array elements
)
docs = loader.load()

# Split as usual
chunks = splitter.split_documents(docs)

# Each chunk is a JSON object as text
for chunk in chunks:
    print(chunk.page_content)
```

- Use `jq` syntax to extract relevant data
- Good for API responses, databases exported as JSON

Cheat Sheet: Common Patterns

```
docs = PyPDFLoader("file.pdf").load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=1000).split_documents(docs)
loader = DirectoryLoader("pdfs/", glob="**/*.pdf",
    loader_cls=PyPDFLoader)
from langchain_community.document_loaders \
  import WebBaseLoader
urls_docs = WebBaseLoader(urls).load()
all_docs = urls_docs + file_docs
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000)
chunks = splitter.split_documents(all_docs)
splitter = TokenTextSplitter(
    encoding_name="cl100k_base", chunk_size=100)
for doc in PyPDFLoader("huge.pdf").lazy_load():
    chunks = splitter.split_documents([doc])
```

Self-Check: Questions 1-3

1. **Document Object:** What are the two main attributes of a LangChain `Document`? Why must metadata be preserved?
2. **Splitting Strategy:** Why does `RecursiveCharacterTextSplitter` work better than simple `CharacterTextSplitter`? Name the separator hierarchy.
3. **Chunk Size:** You're building a RAG system with GPT-4 (128K context). What chunk size would you start with and why? What's the relationship to token limits?

Self-Check: Questions 4-6

4. **Loaders:** Describe the difference between `load()` and `lazy_load()`. When would you use each?

5. **Metadata Pitfall:** What's the difference between `split_text()` and `split_documents()`? Which one preserves metadata?

6. **Large Files:** How would you process a 500-page PDF to avoid running out of memory? Describe the pipeline.

Self-Check: Questions 7-8

7. **Token-Based Splitting:** Why might `TokenTextSplitter` be more accurate than character-based splitting? What encoding names do GPT-4 models use?

8. **Integration:** Write pseudocode for a complete pipeline: load PDFs from `./docs/` directory, split with overlap, embed, and store in a vector database.

Key Takeaways

- **The pipeline matters:** Load → Split → Embed → Store are interdependent
- **Document object is sacred:** Always preserve metadata through all stages
- **RecursiveCharacterTextSplitter is the default:** Use it unless you have a specific reason not to
- **Chunk size affects everything:** Quality of retrieval, cost, speed — experiment with metrics
- **Use lazy_load() for production:** Stream processing avoids memory issues
- **Metadata = traceability:** Without it, you can't cite sources or debug failures

Next Steps & Resources

- **Practice:**
  - Load a PDF, split it, inspect chunks and metadata
  - Experiment with chunk_size and overlap values
  - Build a small RAG prototype with your own documents
- **Resources:**
  - LangChain Docs: `python.langchain.com/docs/modules/data_connection`
  - Token counter: `platform.openai.com/tokenizer`
  - OpenAI context limits: `platform.openai.com/docs/models`
- **Advanced topics:**
  - Semantic chunking (chunk by meaning, not size)
  - Hybrid search (keyword + vector)
  - Document hierarchy and parent-child retrieval

Summary: Quick Reference

Task	Code Pattern	Key Parameter
Load PDF	`PyPDFLoader("file.pdf").load()`	Preserves page #
Load text	`TextLoader("file.txt").load()`	Simple, fast
Load CSV	`CSVLoader("data.csv").load()`	source_column
Load web	`WebBaseLoader([urls]).load()`	Strips HTML
Load dir	`DirectoryLoader("path", glob="**/*.pdf")`	Batch load
Split (default)	`RecursiveCharacterTextSplitter()`	chunk_size, overlap
Split (tokens)	`TokenTextSplitter(chunk_size=100)`	encoding_name
Debug	Print `len(docs)`, inspect `metadata`	Always verify

Questions?

Document Loading & Text Splitting

LangChain v1.2.x — Section 07